

Learning AMG Operators with Graph Neural Networks

AMGLearn with PyMFEM

MFEM Community Workshop – Georgia Tech, Atlanta, GA

Sep 24, 2026



Rui Peng Li (LLNL), Socratis Petrides (LLNL), Kowshik Thopalli (LLNL)

Yousef Saad (UMN), Yuanzhe Xi (Emory), Tianshi Xu (UKY)

Francesco Brarda (Emory), Mitchell Scott (Emory), Soha Yusuf (RPI), Zechen Zhang (UMN)



Existing neural PDE solvers

- **Existing Neural PDE solvers**

- PINNs, DeepONet, FNO, neural operators
- Typically require PDE structure, geometry, discretization information

- **Our goal:** Develop **algebraic** ML-enhanced solvers for

$$Ax = b$$

- Operator design is difficult, heuristic, and highly problem dependent
- **AMGLearn:** Learn AMG operators directly from sparse matrix graphs



- Existing Neural PDE solvers
 - PINNs, DeepONet, FNO, neural operators
 - Typically require PDE structure, geometry, discretization information

- **Our goal:** Develop **algebraic** ML-enhanced solvers for

$$Ax = b$$

- Operator design is difficult, heuristic, and highly problem dependent
- **AMGLearn:** Learn AMG operators directly from sparse matrix graphs



PyMFEM supplies the finite-element problems

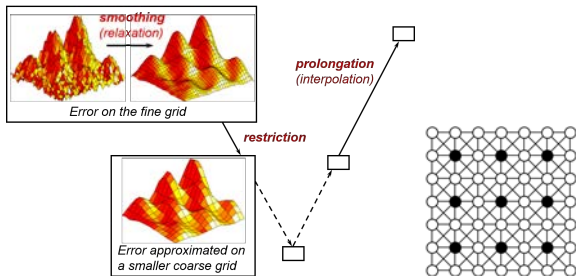
- Build and refine diffusion meshes, define the finite-element space, and assemble the bilinear form.
- Form the linear system as an MFEM `HypreParMatrix`, then export the local sparse matrix as SciPy CSR.
- Remove essential degrees of freedom before creating each learning sample.

PyTorch learns from the assembled operator

- Convert the sparse matrix to a PyTorch Geometric graph and train learned smoothers or transfer operators.
- Keep the MFEM matrix, right-hand side, and boundary-condition maps available for solver evaluation.

PyTorch/PyG training: `mpi4py` is disabled. Use one local assembled matrix.

AMG operators in the multilevel hierarchy



$$v^h := \mathbf{V}^h(A^h, v^h, f^h)$$

Relax $A^h u^h = f^h$
 if $\Omega^h =$ coarsest grid then
 Solve $A^h u^h = f^h$

else
 $f^{2h} := I_h^{2h} (f^h - A^h v^f)$
 $A^{2h} := I_h^{2h} A^h I_{2h}^h$
 $v^{2h} := \mathbf{V}^{2h}(A^{2h}, v^{2h}, f^{2h})$
 $v^h := v^h + I_{2h}^h v^{2h}$
 Relax $A^h u^h = f^h$

- Grid coarsening: $C(\bullet)/F(\circ)$ splitting.
- Two complementary ingredients: relaxation (high frequency modes) and coarse-grid correction (low frequency modes)
- Scalable parallel solver for $Ax = b$ in $\mathcal{O}(n)$ for many applications

AMG error propagation

- Two-grid error propagation:

$$e := \underbrace{(I - M_2^{-1}A)^{\mu_2}}_{\text{post-relaxation}} \underbrace{[I - P(RAP)^{-1}RA]}_{\text{coarse-grid correction (CGC)}} \underbrace{(I - M_1^{-1}A)^{\mu_1}}_{\text{pre-relaxation}} e$$

- M_1, M_2 : smoothers, $P \in \mathbb{R}^{n \times n_c}$: interpolation, $R \in \mathbb{R}^{n_c \times n}$: restriction
- CGC, A -orthogonal projector on $\text{Ran}(P)$, should be able to annihilate the *smooth* modes that cannot be reduced by M
- AMG: let $A_H = RAP$ and recurse. Purely algebraic

IC-type objectives do not predict Krylov convergence

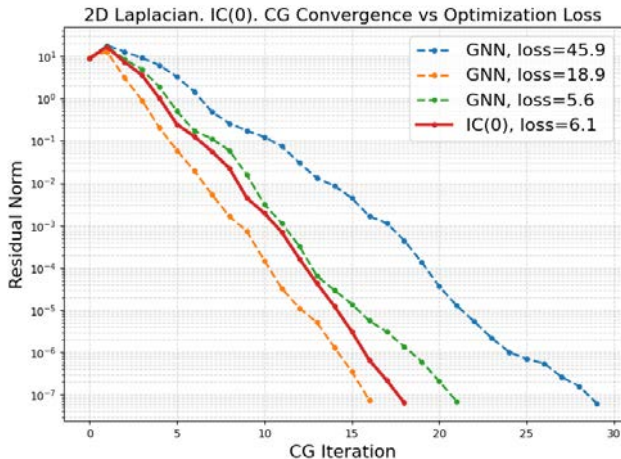
- Neural-network model:
 $G = \text{NN}(A)$
- Performance depends critically on the optimization objective
- IC-type preconditioned matrix:

$$P = G^{-T} A G^{-1}$$

- Classical choice:

$$\|A - G^T G\|_F$$

- Do not directly model Krylov convergence



FSAI-type objectives can fail on indefinite systems

- FSAI-type preconditioned matrix:

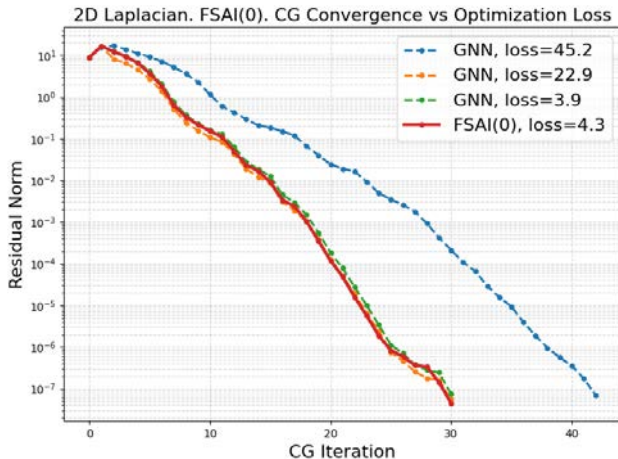
$$P = G^T A G$$

- Classical choice:

$$\|I - P\|_F$$

- Does not apply to indefinite systems

- $\kappa_k(P) = \frac{\text{Tr}(P)/n}{\det(P)^{1/n}}$
- FSAI: $K(G^T A G)$
- K-opt-IC: $K(GA^{-1}G^T)$

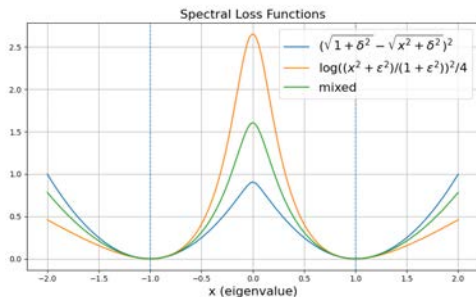


This motivates spectral objectives

Spectral objectives for learning preconditioners/smoothers

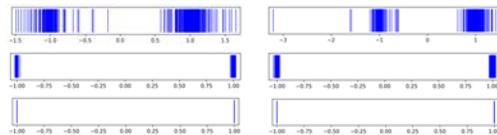
$$\mathcal{L}_f = \frac{1}{n} \sum f(\lambda(P)) = \frac{1}{n} \text{tr}(f(P))$$

$$f(x) = (1 - x)^2 \Leftrightarrow \|I - P\|_F$$



Darcy saddle-point system

$$\begin{cases} ku + \nabla p = f \\ -\nabla \cdot u = g \end{cases}, \quad A = \begin{pmatrix} M & B \\ B^\top & 0 \end{pmatrix}$$



IC. Pattern: A, A^2, A^5 FSAL. Pattern: A, A^2, A^5

MINRES iterations: 6 vs. Schur-AMG: 50

Francesco Brarda, et al. Factored sparse approximate inverse preconditioning via spectral optimization, 2026

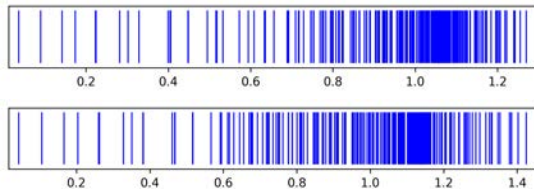
GNN locality and smoother design

- Message-passing GNNs are inherently local, making them well suited for smoother construction
- Robust locally structured smoothers include:

- A standard hard AMG benchmark: rotated diffusion $-\nabla \cdot (QAQ^T \nabla u)$

$$Q = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}, A = \begin{bmatrix} 1 & 0 \\ 0 & \xi \end{bmatrix}$$

- Efficiently damp large eigenmodes

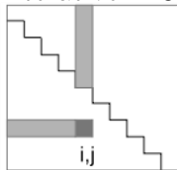


FSAI

$$\begin{matrix} A_{g_i, g_i} \in \mathbb{R}^{m \times m} & g_i \in \mathbb{R}^m & e_m \in \mathbb{R}^m \\ \begin{matrix} j_1 \\ j_2 \\ \vdots \\ j_m \end{matrix} & \begin{bmatrix} g_{i,j_1} \\ g_{i,j_2} \\ \vdots \\ g_{i,j_m} \end{bmatrix} & = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \end{matrix}$$

Kolotilina and Yeremin 1993

Iterative ILU



Chow and Patel 2015

Trifonov, et al. *Can message-passing GNN approximate triangular factorizations of sparse matrices?*, TMLR 2026

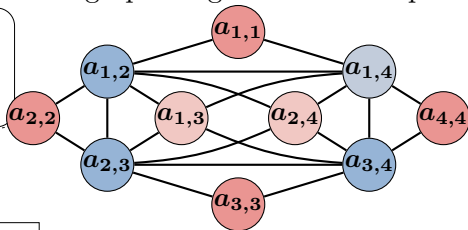
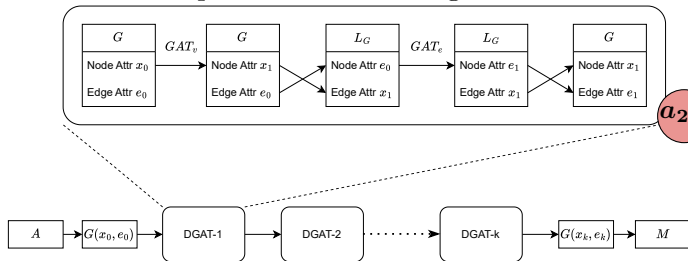
Dual graph attention for matrix operators

Represent the matrix as a graph: variables are nodes, nonzeros are edges, and attention updates both.

Matrix entries live on edges

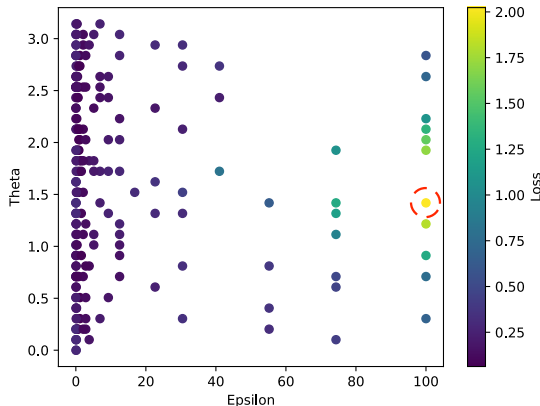
VE-GAT updates node and edge features

Line graph: edge-incidence cliques



Training directly for AMG convergence

- 2D Rotated anisotropic diffusion
- Sample $\xi \in (10^{-2}, 10^2)$ and $\theta \in (0, \pi)$
- GNN uses 5 VEGAT layers and ReLU
- For $\theta = 1.47, \xi = 100$, after fine-tuning, the two-grid convergence factors are
GNN: 0.69; Gauss-Seidel: 0.85
- Loss = two-grid error reduction E_{TG}
- P from classical AMG

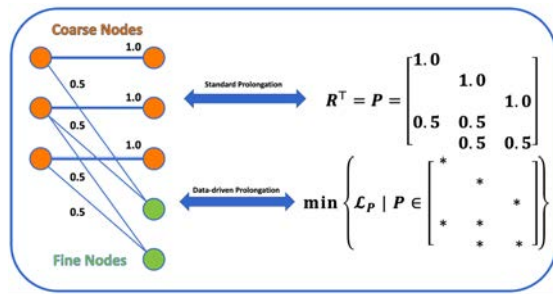


$$E_{TG} = (I - M^{-1}A)(I - P(P^TAP)^{-1}P^TA)$$

Learning AMG interpolation with bipartite GNNs

- Interpolation naturally defines a bipartite structure. The model aggregates information between fine and coarse variables
- Training can use either FEM targets or AMG convergence objectives

$$\mathcal{L}_P = \begin{cases} \|I - M_{TG}^{-1}A\| & \text{Unsupervised} \\ \|P - P_*\| & \text{Supervised} \end{cases}$$



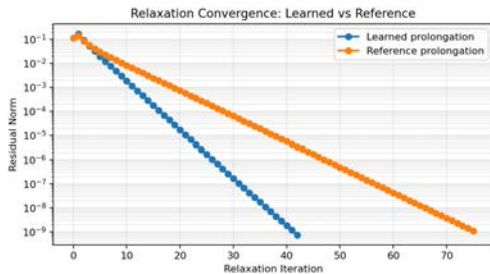
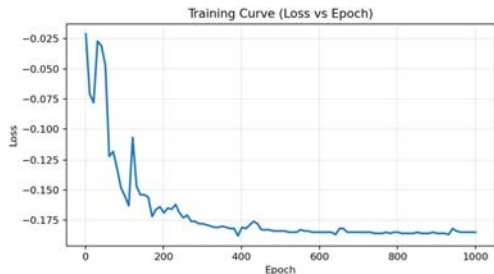
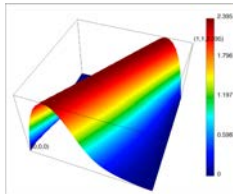
Fix smoother S , $I - M_{TG}^{-1}A = (I - S^{-1}A)(I - P(P^TAP)^{-1}P^TA)(I - S^{-1}A)$

Friday. Mitchell Scott (Emory) “Accelerating multigrid prolongation via bipartite graph attention networks”

Learned interpolation on rotated anisotropic diffusion

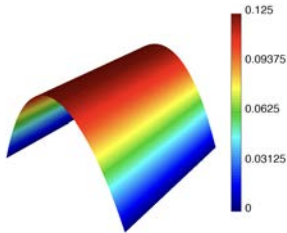
$$\begin{cases} -\nabla \cdot (K(x) \nabla u(x)) = f(x), & \text{in } \Omega, \\ u(x) = 0, & \text{on } \Gamma_d, \\ (K(x) \nabla u(x)) \cdot n = 0, & \text{on } \Gamma_N, \end{cases}$$

$$K(x) = R(\theta) \begin{pmatrix} 1 & 0 \\ 0 & \epsilon \end{pmatrix} R(\theta)^T, \quad R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}, \quad \epsilon = 0.01, \theta = \pi/3.$$

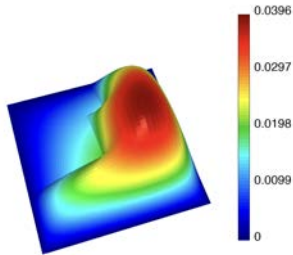


Jointly learning two-grid interpolation and smoothing

- Interpolation pattern follows the sparsity of $\mathbf{A}$ (GMG pattern for order 1) or AMG.
- GNN learns weights: learn $\mathbf{P}$ with fixed damped Jacobi, or jointly learn $\mathbf{P}$ and $\mathbf{M}$.
- Convergence-based loss: use the power method to estimate the spectral radius of the two-grid iteration matrix.

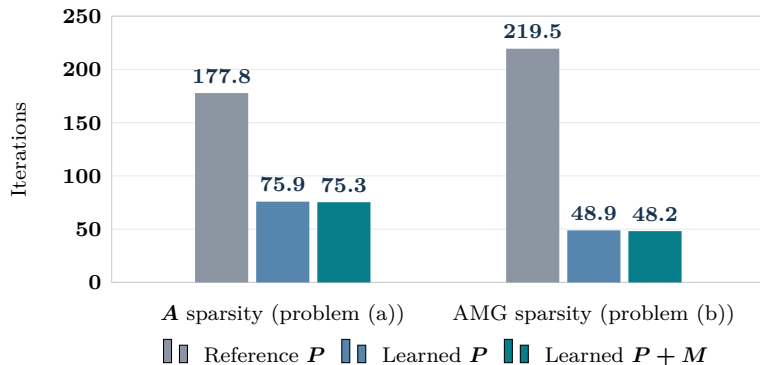


(a) $K = \text{diag}(1, 0.01)$



(b) $K(x) = k(x)I$, with $k = 10$ in three subregions, 1 elsewhere

Selected results with A and AMG sparsity



58% fewer

Two-grid iterations, A pattern
177.80 vs. 75.30

78% fewer

Two-grid iterations, AMG pattern
219.45 vs. 48.15

Reference: GMG P for (a), AMG P for (b), both with fixed Jacobi.

Reinforcement learning for AMG coarsening with GNNs

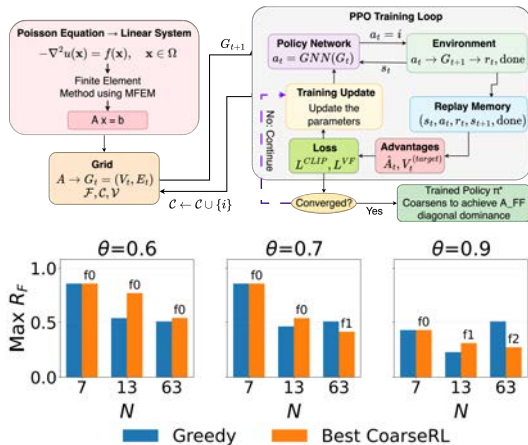
- For the C/F splitting $\begin{pmatrix} \mathbf{A}_{FF} & \mathbf{A}_{FC} \\ \mathbf{A}_{CF} & \mathbf{A}_{CC} \end{pmatrix}$

(1) $\mathbf{A}_{FF}$ satisfies the diagonal-dominance (DD) threshold θ_d

(2) $\mathbf{R}_F = |\mathbf{F}|/\mathbf{N}$ is as large as possible

- Classical **greedy** heuristics
- A sequential combinatorial Markov decision process (MDP)

- RL algorithms: Dueling DQN, PPO
- GNN: TAGConv, GCN
- Step penalties and terminal rewards



Sources: MacLachlan and Saad (2007), Taghibakhshi et al. (2021), Yusuf et al. (2026)

Graph neural multilevel preconditioners (GMP): cross-attention

- AMG interpolation:

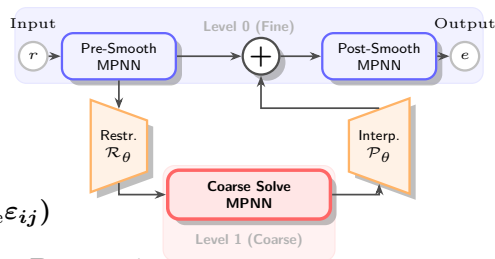
$$(Pv)_i = \sum_{j \in C_i} w_{ij} v_j, \quad i \in \mathcal{V}_F$$

- Replace w_{ij} with *attention weights*:

$$h_i = \Theta_{\text{self}} h_i + \sum_{j \in C_i} \alpha_{i,j} (\Theta_{\text{val}} h_j + \Theta_{\text{edge}} \epsilon_{ij})$$

Attention-based interpolation:

- **Queries:** F-nodes (to correct)
- **Keys/Values:** C-nodes (provide info)
- **Attention score:** $\alpha_{i,j}$



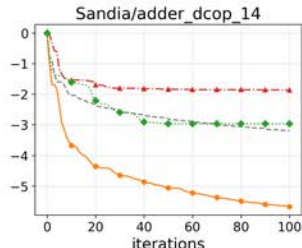
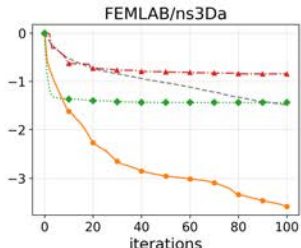
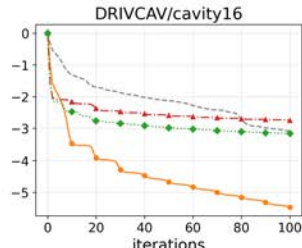
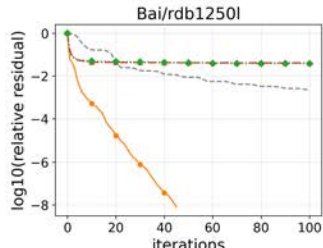
Properties:

- ✓ ℓ^∞ -stable and preserves constants
- ✓ Adapts to local structure and the iterate
- ✓ Generalizes BAMG least squares

Zhang, Li, Saad, “Graph Neural Multilevel Preconditioners for Iterative Solvers”, KDD’26 AIS

For non-SPD systems: GMP continues reducing the residual after AMG and AIR stall

- 867 real, square, non-SPD matrices from SuiteSparse
- 1,000–100,000 unknowns and fewer than 2 million nonzeros
- FGMRES stops at relative residual 10^{-8} or 100 iterations
- PyTorch/PyG on one NVIDIA Tesla A100 (40 GB)



Gray: no preconditioner Orange: GMP Red: AMG Green: AIR

Conclusions and future directions

1. AMGLearn learns smoothers, transfer operators, and coarsening while retaining the AMG hierarchy.
2. Learned operators can improve residual reduction and robustness when classical heuristics struggle.
3. PyMFEM provides the finite-element workflow; future work jointly optimizes all multilevel operators.

THANK YOU!

Questions & Comments

✉ li50@llnl.gov

